

Cryptographic Primitive Benchmarking Under Cycle-Accurate Emulation for Nano-Bee Secure Communications

Week 4 Research Proposal

VESPER-SEC-01 Project

Domain Track: Cybersecurity (Option 3.5 A)

Subsystem Tier: B-System · Mothership

VISVAMBHARA (Weeks 4–12)

Rutvika Mahadev Ashtekar

September 2026

Abstract

Autonomous swarm agents at the Nano-Bee tier must authenticate and encrypt command and telemetry traffic exchanged with the B-System, but operate under severe compute, memory, and energy constraints. This project presents a zero-cost, emulation-based benchmarking framework that will measure the runtime cost of four representative cryptographic primitives — a NIST-standardized lightweight cipher, a proven symmetric AEAD scheme, and two NIST-standardized post-quantum primitives — on an ARM Cortex-M4 target under Renode cycle-accurate emulation. The framework will instrument each primitive via the Cortex-M4 Data Watchpoint and Trace (DWT) cycle counter to obtain cycles/byte, peak RAM footprint, and code size, and will derive an energy-per-operation estimate from measured cycle counts using the target chip's datasheet-published current-consumption figure. As of this Week 4 submission, the toolchain (Arm GNU Toolchain, QEMU, Renode) is installed, its **functional correctness** has been confirmed by successfully loading and executing real Cortex-M4 firmware, and a **trivial cycle-accuracy check** (1,000 NOP instructions, measured against the ARM Cortex-M4 Technical Reference Manual's published instruction timing) has been completed with a 0.3% discrepancy, well within the stated tolerance. A **cryptography-specific cycle-accuracy comparison** against a published result (Heinz, masked Kyber-512 on Cortex-M4) remains outstanding and is reported honestly as the next step, per the charter's four-way verification discipline. The resulting comparison table and feasibility verdict will directly inform which primitives, if any, the Nano-Bee secure-communications subsystem can deploy within its stated resource budget of 18 MHz compute (≈ 14.1 mW at 3.3 V) and 50 KB RAM.

Keywords: Lightweight Cryptography, Post-Quantum Cryptography, ARM Cortex-M4, Cycle-Accurate Emulation, Embedded Security, Nano-Bee Secure Communications.

1. Introduction

Secure communication between the Mothership, B-System, and individual Nano-Bee agents requires authenticated encryption capable of resisting both classical and, eventually, quantum-capable adversaries. Unlike server or desktop deployments, the Nano-Bee tier runs on a single low-power ARM Cortex-M4 microcontroller with a tightly bounded compute, memory, and power allocation, since the majority of onboard resources are reserved for flight control and mission payload processing.

Standard cryptographic libraries are benchmarked almost exclusively on high-end platforms. NIST's Lightweight Cryptography Standardization process (concluding with Ascon as the selected standard) and

the NIST Post-Quantum Cryptography Standardization process (concluding with ML-KEM and SPHINCS+ as standards) provide candidate algorithms explicitly designed for constrained environments, but their concrete cycle-level performance on the Nano-Bee's specific target configuration remains unquantified.

2. Problem Statement

The Nano-Bee target platform is modelled on an STM32F407-class ARM Cortex-M4 microcontroller operating at up to 168 MHz with 192 KB of SRAM (plus a separate 4 KB backup SRAM) — confirmed from ST's official datasheet (DS8626). Current subsystem-level engineering estimates (*an assumption made for this proposal, not a value from an external Nano-Bee specification*) reserve 18 MHz of the 168 MHz total clock for the secure-communications subsystem, with the remainder allocated to flight control, sensor fusion, and payload processing. Using the STM32F407's datasheet-published run-mode current figure of 238 $\mu\text{A}/\text{MHz}$ at full clock [18], this 18 MHz allocation corresponds to a derived power budget of approximately 14.1 mW at 3.3 V supply ($238 \mu\text{A}/\text{MHz} \times 18 \text{ MHz} \times 3.3 \text{ V}$), alongside a reserved 50 KB of the chip's 192 KB SRAM (*also an assumed reservation, stated explicitly so it can be revised if a shared programme-wide Nano-Bee spec becomes available*). Whether any standard cryptographic primitive — lightweight, classical symmetric, or post-quantum — can operate within this residual budget is presently unknown.

Physical hardware profiling is unavailable under the charter's remote, zero-cost constraints. This project instead establishes feasibility through cycle-accurate emulation, instrumented in the same way physical hardware would be measured, via the Cortex-M4's built-in cycle counter.

Figure 1: Emulation-Based Crypto Benchmarking Pipeline (VESPER-SEC-01)

- 1. Reference Implementation & Compilation** — `arm-none-eabi-gcc, -mcpu=cortex-m4 -mthumb`
- 2. Emulated Platform Configuration** — Renode: STM32F4 Discovery board model, cycle-accurate core
- 3. Cycle-Accurate Timing** — `DWT->CYCCNT` instrumentation per primitive
- 4. Memory & Code-Size Profiling** — `stack high-water mark + arm-none-eabi-size`
- 5. Energy Estimation & Feasibility Verdict** — `cycles → energy → budget comparison`

3. Executive Alignment & Subsystem Impact

In accordance with VISVAMBHARA charter conventions, the contribution of this project is defined as follows:

“If this project succeeds, the Nano-Bee secure-communications subsystem improves by providing a quantified go/no-go verdict on which cryptographic primitives can be deployed within the platform's compute, memory, and power constraints — measured by a feasibility matrix determining which primitives fit the stated 18 MHz / 50 KB / 14.1 mW budget.”

The project adheres to the three charter constraints: (1) zero software cost using open-source Renode, QEMU, and the Arm GNU Toolchain; (2) ordinary laptop execution (8 GB RAM, no discrete GPU); and (3) no laboratory or physical hardware, replacing bench measurement with cycle-accurate emulation and instruction-timing cross-checks against published ARM documentation.

4. Structured Literature Review

The theoretical and empirical foundation draws on 18 references across five domains.

4.1 Lightweight Authenticated Encryption

Dobraunig et al. [1] specify Ascon v1.2, the NIST-selected lightweight authenticated encryption and hashing standard. Banik et al. [2] introduce GIFT, a compact block cipher family used as an LWC comparison baseline. Wu and Huang [3] describe TinyJAMBU, a further NIST LWC finalist optimized for minimal gate/RAM footprint.

4.2 Symmetric & Stream Ciphers

Nir and Langley [4] standardize ChaCha20-Poly1305 as an IETF AEAD scheme (RFC 8439). Bernstein [5] defines the underlying ChaCha stream cipher. NIST [6] specifies the Advanced Encryption Standard (FIPS PUB 197), used as the classical symmetric baseline. Cruz et al. [7] provide an early empirical study of lightweight cryptography performance specifically on ARM targets.

4.3 Post-Quantum Cryptography

Heinz [8] presents cycle-level Cortex-M4 timing for a masked Kyber implementation — the primary reference this project's cryptography-specific cycle-accuracy check will be validated against. Bos et al. [9] introduce CRYSTALS-Kyber's design rationale. Bernstein et al. [10] define the SPHINCS+ stateless signature framework. NIST [11] formally standardizes ML-KEM (FIPS 203). Abdulrahman et al. [12] and Botros et al. [13] provide optimized, Cortex-M4-specific Kyber and Dilithium implementations and their measured cycle counts.

4.4 Embedded Hardware & Emulation Toolchain

Yiu [14] documents the Cortex-M4 instruction set, pipeline, and memory architecture forming the hardware baseline for all timing analysis, including the instruction-timing table used for this proposal's trivial cycle-accuracy check (Section 8.3). Antmicro [15] and the QEMU Project [16] document the Renode and QEMU emulation environments respectively. Arm Limited [17] documents the GNU Toolchain used for cross-compilation.

4.5 Target Hardware Specification

STMicroelectronics [18] publishes the STM32F407/417 product datasheet and current-consumption figures used throughout this project's power budget derivation, including the 238 μ A/MHz run-mode current at 168 MHz used in Section 5.5 and Section 2 above.

5. Technical Methodology

The benchmarking framework will operate in five sequential stages. **As of this Week 4 submission, the toolchain installation, a functional-execution check (Stage 2), and a trivial instruction-timing verification (Stage 3, partially) have all been completed — see Section 8 for exactly what has and has not been done. Full cryptographic cycle-accurate benchmarking across all four primitives begins in Week 6.**

5.1 Stage 1: Reference Implementation Selection & Compilation

Reference C implementations of each primitive (Ascon, ChaCha20-Poly1305, ML-KEM/Kyber-512, SPHINCS+) will be compiled with arm-none-eabi-gcc targeting Cortex-M4 (-mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -O2), matching the flags used in published Cortex-M4 benchmarking studies [8, 12, 13] to keep results comparable.

5.2 Stage 2: Emulated Platform Configuration

Renode instantiates an STM32F4 Discovery board model (Cortex-M4, 168 MHz, 192 KB SRAM), providing cycle-accurate core execution and peripheral timing without physical hardware. This stage has been completed: the STM32F4 Discovery machine model has been successfully instantiated and its reference demo firmware executed under Renode (see Section 8.2).

5.3 Stage 3: Cycle-Accurate Timing

Each primitive will be instrumented using the Cortex-M4 Data Watchpoint and Trace (DWT) cycle counter:

$$DWT->CYCCNT \leftarrow 0, \quad DWT->CTRL |= CYCCNTENA \quad (1)$$

Cycles per byte for a message of length L bytes will be computed as:

$$C_{\text{byte}} = C_{\text{op}} / L \quad (2)$$

where C_{op} (C_{op}) is the measured cycle count for one complete encryption/decryption (or encapsulation) operation. A trivial version of this methodology (instruction count rather than the full DWT register) has already been verified for a fixed, known instruction sequence — see Section 8.3.

5.4 Stage 4: Memory & Code-Size Profiling

Peak RAM usage will be obtained via stack high-water-mark measurement (fill-pattern method), and static code size via `arm-none-eabi-size` on the compiled binary.

5.5 Stage 5: Energy Estimation & Feasibility Verdict

Energy per operation will be estimated from measured cycles using the STM32F407's datasheet-published current-consumption figure of 238 $\mu\text{A}/\text{MHz}$ at 168 MHz, running from Flash [18]:

$$E_{\text{op}} = (C_{\text{op}} / f_{\text{clk}}) \times V_{\text{dd}} \times (238 \mu\text{A}/\text{MHz} \times f_{\text{clk},\text{MHz}}) \quad (3)$$

where C_{op} is the measured cycle count for one operation, $f_{\text{clk}} = 168 \text{ MHz}$, and $V_{\text{dd}} = 3.3 \text{ V}$. At the reserved 18 MHz allocation, this yields an available power budget of approximately 14.1 mW. A primitive will be classified FEASIBLE if $C_{\text{p}}^{\text{yte}}$, peak RAM, and E_{op} simultaneously satisfy this budget; MARGINAL if within 25% of any single limit; otherwise NOT FEASIBLE.

6. Quantified Success Metrics

Project progress will be evaluated against three quantifiable metrics:

- 1. Toolchain Verification Accuracy:** measured Renode cycle count for a known instruction sequence within $\leq 5\%$ of the value predicted by the ARM Cortex-M4 Technical Reference Manual's published instruction-timing table. (*Trivial case completed — 0.3% discrepancy, see Section 8.3. Cryptography-specific case, against Heinz [8], pending — see Section 8.4.*)
- 2. Benchmark Coverage:** ≥ 4 primitives (Ascon, ChaCha20-Poly1305, ML-KEM/Kyber-512, SPHINCS+) fully characterized across cycles/byte, peak RAM, code size, and estimated energy.
- 3. Feasibility Verdict:** a complete comparison table stating which primitives satisfy $\leq 18 \text{ MHz}$ effective compute load ($\approx 14.1 \text{ mW}$ at 3.3 V) and $\leq 50 \text{ KB}$ RAM.

7. Risk Assessment and Mitigation

Major risks and planned technical mitigations are summarized in Table 1.

Table 1: Risk Assessment and Mitigation Strategy

Risk Factor	Severity	Mitigation Strategy
-------------	----------	---------------------

Emulator cycle-count inaccuracy	High	Validate against Heinz (2021) published Kyber-512 Cortex-M4 timing before scaling to remaining primitives. If discrepancy exceeds 10%, re-tune Renode settings or fall back to QEMU.
Post-quantum code-size / cycle explosion	Medium	Expected outcome, not a failure. Document as an infeasibility finding and report the mechanism (lattice/hash-tree overhead).
Missing reference implementations for target	Low	pqm4 and PQClean repositories provide Cortex-M4-ready reference code as fallback.

8. Toolchain Verification

This section reports exactly what has been confirmed and what remains outstanding, in accordance with the charter's requirement that a toolchain be verified — not merely installed — before it is trusted for cryptographic measurement. Screenshot evidence for every step below is provided separately (evidence pack / project repository, verification/screenshots/).

8.1 Installation Confirmation

All three required tools were confirmed installed via direct version checks in the local development environment (Windows / PowerShell):

```
arm-none-eabi-gcc.exe (Arm GNU Toolchain 12.2.MPACBTI-Rel1) 12.2.1
QEMU emulator version 11.1.0 (v11.1.0-12130-ge470268ff4)
Renode, version 1.16.0 (20ad06d9-202508030343)
```

The Arm GNU Toolchain build includes MPACBTI support (Pointer Authentication and Branch Target Identification), a hardware-enforced ARM extension for Cortex-M that mitigates Return-Oriented and Jump-Oriented Programming (ROP/JOP) attacks during binary execution — noted here as a relevant security property of the target build, though not itself exercised by this project's benchmarking work.

Evidence: gcc_version.png, qemu_version.png, renode_version.png

8.2 Functional Execution Verification

Beyond installation, functional correctness was verified by running Renode's built-in reference demonstration script for the STM32F4 Discovery board (stm32f4_discovery.resc), then issuing start. This loaded the reference firmware (78,080 bytes at address 0x80000000 and 25,952 bytes at address 0x8013100) into the emulated STM32F4Discovery machine and began emulation. The resulting log showed a continuous stream of real, timestamped peripheral register accesses (SYSCFG, Ethernet PHY, and Ethernet MAC) at correct memory-mapped addresses — for example, reads and writes to SYSCFG:EXTICR3 at 0x40013808 and Ethernet PHY register writes at documented offsets. This confirms the emulated CPU is genuinely executing loaded Cortex-M4 instructions rather than remaining idle or producing placeholder output. A small number of “unimplemented register” warnings appeared during this run; these are expected and normal — Renode's STM32F4 peripheral model does not implement every register's exact behaviour, and returns a default value rather than failing when firmware touches one it has not modelled. This does not affect the validity of the execution proof.

Evidence: renode_execution_proof.png

8.3 Cycle-Accuracy Verification (Trivial Case) — Completed

A minimal bare-metal firmware image consisting of 1,000 NOP instructions followed by an infinite branch loop was compiled (arm-none-eabi-gcc, -mcpu=cortex-m4 -mthumb) and loaded into the same STM32F4 Discovery machine verified in Section 8.2. Renode's built-in instruction counter was read before and after execution:

```
Before execution: ExecutedInstructions = 0x0000000000000000 (0)
After execution: ExecutedInstructions = 0x00000000000003EB (1003)
```

Since NOP is documented as a single-cycle instruction on Cortex-M4 per the ARM Cortex-M4 Technical Reference Manual [14], the expected count for 1,000 NOPs is 1000. The measured count of 1003 gives a discrepancy of 3 instructions (0.3%), well within the $\leq 5\%$ tolerance stated in Section 6. The 3-instruction excess is consistent with the small number of additional instructions executed while entering the reset handler and beginning the trailing infinite branch loop before emulation was paused.

This result validates Renode's instruction-execution accuracy for this trivial case. Note that this test measures instruction count, which is equivalent to cycle count here only because NOP is a fixed 1-cycle operation; it does not yet exercise the DWT-based CYCCNT cycle-counter methodology required for multi-cycle, branching cryptographic code (Section 5.3).

Evidence: renode_cycle_verification.png

8.4 Outstanding: Cryptography-Specific Cycle-Accuracy Validation

This remains NOT YET COMPLETE. The trivial NOP-based check above (Section 8.3) validates instruction-execution accuracy for a single-cycle, non-branching case. It does not yet validate cycle-accurate timing for real, multi-cycle, branching cryptographic code. The charter's four-way verification discipline (§1.1, VESPER-TASK-W412-002) requires this cryptography-specific reproduction before any cycle-count result from a crypto primitive is trusted. The planned method, per the Risk 1 mitigation in Table 1, is:

1. Enable the Cortex-M4 DWT cycle counter (CoreDebug->DEMCR, DWT->CYCCNT, DWT->CTRL) in a minimal bare-metal firmware image.
2. Compile and run a reference Kyber-512 implementation matching Heinz [8]'s tested configuration.
3. Compare the measured DWT cycle count against Heinz [8]'s published Cortex-M4 timing.
4. If discrepancy exceeds 10%, re-tune Renode settings or fall back to QEMU, per Table 1.

Until this step is complete, no cryptographic cycle-count number produced by this toolchain should be treated as validated. This is reported honestly rather than presented as done, consistent with the charter's stated preference for an understood, honestly-reported gap over an unverified claim. This is planned as the first task of Week 5/6, once primitive benchmarking begins.

9. Test Environment

All work in this project is performed entirely within Renode, emulating an STM32F4 Discovery board (ARM Cortex-M4). No physical hardware is used. No external network targets, third-party systems, or devices are tested against. All execution occurs locally on the researcher's own machine (Windows, PowerShell), consistent with the Cybersecurity domain's scope boundary in the charter (§3.5).

10. Conclusion

This Week 4 proposal establishes a complete, defensible experimental roadmap for VESPER-SEC-01. The toolchain — Arm GNU Toolchain, QEMU, and Renode — is installed, its functional correctness has been

demonstrated by successfully executing real Cortex-M4 reference firmware, and a trivial cycle-accuracy check (1,000 NOP instructions) has been completed with a 0.3% discrepancy against the ARM Cortex-M4 Technical Reference Manual's published timing, well within the $\leq 5\%$ tolerance stated in Section 6. The remaining validation step is a cryptography-specific cycle-accuracy comparison against Heinz [8]'s published Kyber-512 Cortex-M4 timing, planned via the same DWT cycle-counter methodology (Section 5.3) at the start of Week 5/6. Once verified, the same instrumentation will be extended across four NIST-standardized cryptographic primitives (Ascon, ChaCha20-Poly1305, ML-KEM/Kyber-512, SPHINCS+) to produce a quantified feasibility verdict for Nano-Bee secure-communications deployment under the derived 18 MHz / 50 KB / 14.1 mW budget, consistent with the charter's zero-cost, remote-research format.

Planned Cross-Domain Integration (Week 9)

Per the charter's integration table (§5), this project's eventual output — a benchmarked crypto primitive and its measured timing cost — is designated for exchange with the Embedded Software & Real-Time Systems domain (VESPER-SWE-xx), where it will inform secure-boot integration cost calculations.

References

- [1] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schl  ffer, "Ascon v1.2: Lightweight Authenticated Encryption and Hashing," 2021.
- [2] S. Banik, S. Pandey, T. Peyrin, Y. Sasaki, S. M. Sim, and Y. Todo, "GIFT: A Small Present," in Cryptographers' Track at the RSA Conference, Springer, 2017, pp. 321–345.
- [3] H. Wu and T. Huang, "TinyJAMBU: A Family of Lightweight Authenticated Encryption Algorithms," NIST Lightweight Cryptography Standardization Process, 2021.
- [4] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC 8439, IETF, 2018.
- [5] D. J. Bernstein, "ChaCha, a variant of Salsa20," in Fast Software Encryption, 2006.
- [6] National Institute of Standards and Technology, "FIPS PUB 197: Advanced Encryption Standard (AES)," U.S. Department of Commerce, 2001.
- [7] R. J. Cruz, T. B. Reis, D. F. Aranha, J. L  pez, and H. K. Patil, "Lightweight Cryptography on ARM," NIST Lightweight Cryptography Workshop, 2016.
- [8] M. Heinz, "First-Order Masked Kyber on ARM Cortex-M4," NIST CSRC PQC Workshop Presentation, 2021.
- [9] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schanck, P. Schwabe, G. Seiler, and D. Stehl  , "CRYSTALS – Kyber: A CCA-secure module-lattice-based KEM," in IEEE European Symposium on Security and Privacy (EuroS&P), 2018, pp. 353–367.
- [10] D. J. Bernstein, A. H  lsing, S. K  lbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, "The SPHINCS+ Signature Framework," NIST PQC Standardization Round 2 Submission, 2019.
- [11] National Institute of Standards and Technology, "FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)," 2024.
- [12] A. Abdulrahman, V. Hwang, M. J. Kannwischer, and A. Sprenkels, "Faster Kyber and Dilithium on the Cortex-M4," in Applied Cryptography and Network Security (ACNS), 2022, pp. 853–871.
- [13] L. Botros, M. J. Kannwischer, and P. Schwabe, "Memory-Efficient High-Speed Implementation of Kyber on Cortex-M4," in Applied Cryptography and Network Security (ACNS), 2019, pp. 209–228.
- [14] J. Yiu, The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd ed. Newnes, 2013.
- [15] Antmicro, "Renode: Open Source Multi-Node Simulator," 2026. Available: <https://renode.io/>
- [16] QEMU Project Developers, "QEMU System Emulation User Guide: ARM Targets," 2025. Available: <https://www.qemu.org/docs/master/system/target-arm.html>
- [17] Arm Limited, "Arm GNU Toolchain for embedded 32-bit RISC processors," 2025. Available: <https://developer.arm.com/Tools%20and%20Software/GNU%20Toolchain>
- [18] STMicroelectronics, "STM32F407/417 — Arm Cortex-M4 32-bit MCU," product page, 2026. Available: <https://www.st.com/en/microcontrollers-microprocessors/stm32f407-417.html>